华睿工业相机 SDK 开发手册

（C#篇）

Ver1.2

变更履历

编号	变更日期	版本号	变更内容概要	配套 SDK 版本
1	2023/03	V1.0	更新内容	2.4.0
2	2024/04	V1.1	增加保存图片接口	2.4.2
3	2025/11	V1.2	增加 MVPImage 构造方式	2.6.1

目录

1. SDK 概要说明 1
1.1. SDK 支持的操作系统 1
1.2. SDK 安装目录 1
1.2.1. Windows 版 SDK 1
1.2.2. Linux 版 SDK 3
1.3. SDK（C#接口）对应的资源 4
1.4. SDK 开发工程配置 4
1.4.1. VS2013 ~ VS2022 4
2. SDK 的整体调用流程 8
3. SDK 的各功能接口调用说明 9
3.1. 系统操作 9
3.1.1. 获取版本信息 9
3.1.2. 枚举设备（发现设备） 9
3.1.3. 设备句柄的创建与销毁 9
3.2. 相机操作 10
3.2.1 相机的连接与关闭 10
3.2.2 相机配置的下载、保存及加载配置 11
3.2.3 相机配置设置（GigE 相机特有） 11
3.2.4 本品牌相机特有功能 12
3.3. 注册事件回调 13
3.4. 相机流资源操作 14
3.4.1. 流数据设置 14
3.4.2. 图像采集 15
3.4.3. 获取图像数据 15
3.4.4. 流与图像数据的其他操作 16
3.5. 属性操作 17
3.5.1. 属性的判断、获取、设置 17
3.5.2. 命令属性的执行 21
3.5.3. 属性的查询与使用 21
3.6 图像操作 29
3.6.1 录像相关 29
3.6.2 图像相关 30
4. 第三方平台的图像对象转换方法 31
4.1. Halcon 对象 31
4.1.1. 相机图像格式为 Mono8 时（主要是黑白格式） 31
4.1.2. 相机图像格式为 Mono8 以外时（主要是彩色格式） 31
4.2. Mvp 图像对象（MvpImage） 32
4.2.1. 相机图像格式为 Mono8 时 32
4.2.2. 相机图像格式为 Mono8 以外时（主要是彩色格式） 32
5. 配套例程说明 33
ChunkData 33
6. 常见问题&注意点 34

6.1. 客户的程序使用 C#接口采图，只能取到 8 张图像（和 SDK 缓存个数一致）。..34

1. SDK 概要说明

1.1. SDK 支持的操作系统

SDK 支持以下版本的操作系统

操作系统类型	操作系统版本号	备注
Windows *1	Win7 SP1 32/64bit	务必注意带 SP1 补丁
	Win8 32/64bit	
	Win10 32/64bit	
Linux (x86) *2	系统版本： Ubuntu 12.04 LTS ~ Ubuntu 21.04 LTS （32/64bit）	不包括升级了内核版本的系统
	CentOS 6.5 ~ CentOS 8.0 （32/64bit）	不包括升级了内核版本的系统

*1. 从 SDK2.2.5 开始已经不支持 windows xp 系统

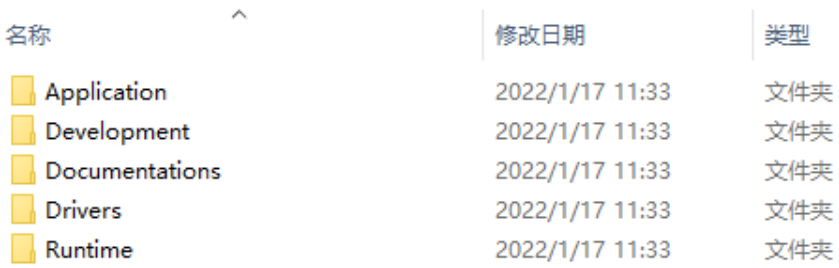
*2. 目前支持的 Linux 内核版本号从 2.6.32（含）到 5.11.0（含）。
可以通过 uname –a 命令来确定系统内核版本号。

1.2. SDK 安装目录

1.2.1. Windows 版 SDK

Windows 版本 SDK 的安装目录结构如下。

※ 从 Ver2.1.0 版本开始，用户可以自己选择安装位置。



文件夹说明如下：

Application	1.相机客户端软件，包含了 32 位与 64 位 2.相机配套工具软件，包含了 32 位与 64 位
Development	**3rdPartyPlatformAdapter** 包含了两个第三方算法平台（Halcon、Sherlock）插件动态库
	DOTENT DLL 包含.Net 库文件，在创建项目时从中导入
	Include 包含了 SDK 的接口头文件，头文件中解释了各个接口的基础属性，具体用法在 samples 文件夹中有示例 另一个是 SDK 扩展定义头文件，定义了 SDK 的错误码、参数自定义类型、些许参数的数据限制、及各类行为方式的枚举、定义了多种信息的数据结构
	Lib 包含 lib 库文件
	Samples 包含 demo 示例（C#、Delphi、Lab View、MFC、Python、QT、VB、VC）

Documentations	.Net 接口文档 ActiveX 接口文档 C 接口文档 SDK 开发手册
Drivers	DShowFilter 的安装与卸载程序 GigE 高性能驱动的安装与卸载程序 PCIex86 智能相机驱动的安装与卸载程序 usb 高性能驱动的安装与卸载程序
Runtime	SDK 运行时编译动态链接库 包含第三方 GenlCam 库 分为 32 位与 64 位两个文件夹

1.2.2. Linux 版 SDK

Linux 版本 SDK 的文件夹结构、以及概要说明如下：

```
├ bin              : GUI工具，发现、连接相机，拉流取图，设置相机属性，卸载脚本，etc
├ include          : SDK头文件、转码头文件
├ lib              : SDK动态库、第三方库、转码库
├ module
|  ├ GigEDriver   : GigE相机驱动程序
|  └ USBDriver    : USB相机的驱动程序
└ share
   ├ C             : C例程
   ├ Qt            : Qt例程
   └ Python        : Python例程
```

1.3. SDK（C#接口）对应的资源

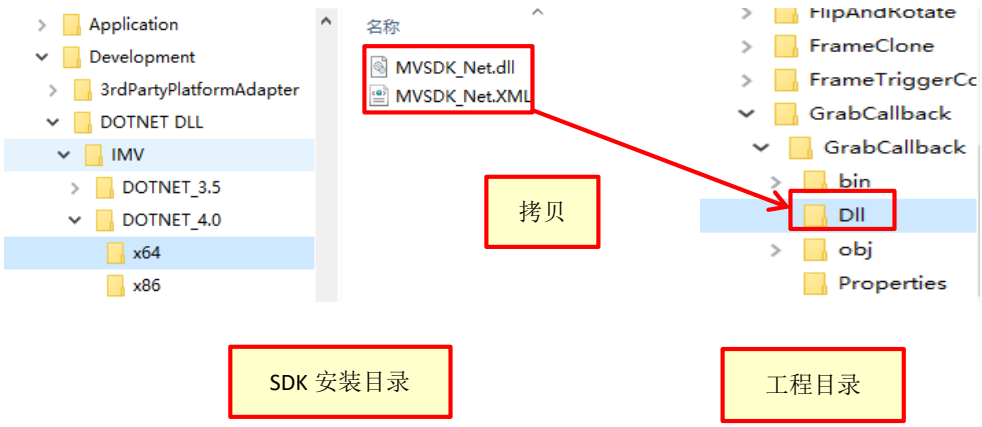
大类别	小类别	文件位置	文件名
SDK 库	DOTNET_3.5（32 位）	【SDK 安装目录】\Development\DOTNET DLL\IMV\DOTNET_3.5\x86	MVSDK_Net.dll
	DOTNET_3.5（64 位）	【SDK 安装目录】\Development\DOTNET DLL\IMV\DOTNET_3.5\x64	MVSDK_Net.dll
	DOTNET_4.0（32 位）	【SDK 安装目录】\Development\DOTNET DLL\IMV\DOTNET_4.0\x86	MVSDK_Net.dll
	DOTNET_4.0（64 位）	【SDK 安装目录】\Development\DOTNET DLL\IMV\DOTNET_4.0\x64	MVSDK_Net.dll
开发文档	中文	【SDK 安装目录】\Documentations\	IMV_API_Net_CHS.chm 工业相机 SDK 开发手册(C#篇).pdf
	英文	【SDK 安装目录】\Documentations\	IMV_API_Net_ENG.chm
开发例程	\	【SDK 安装目录】\Development\Sample\C#\IMV	

1.4. SDK 开发工程配置

1.4.1. VS2013 ~ VS2022

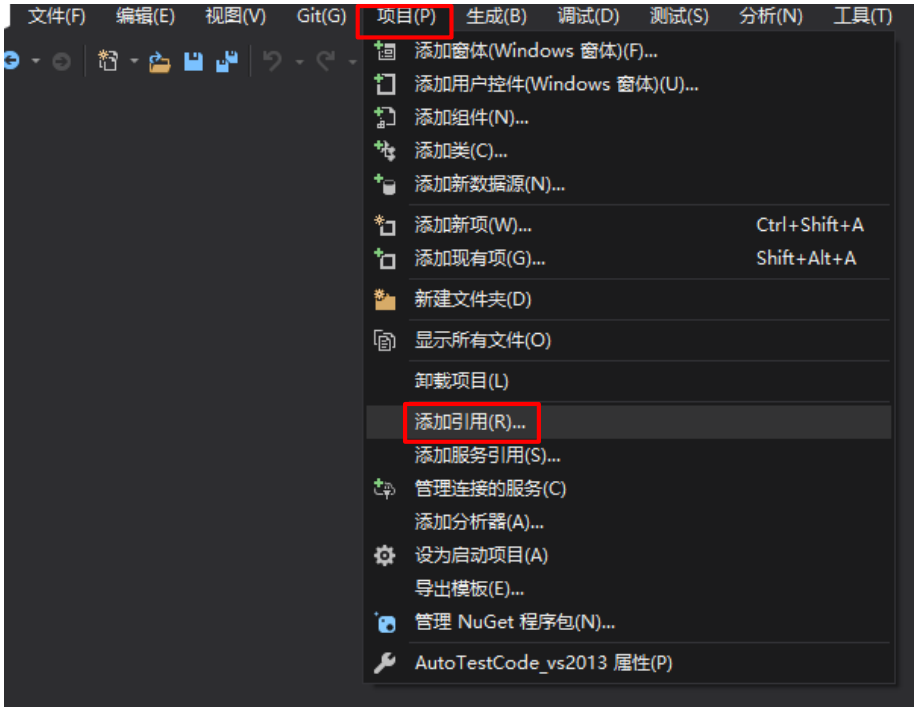
下面以 DOTNET4.0 64 位为例演示工程配置方法。

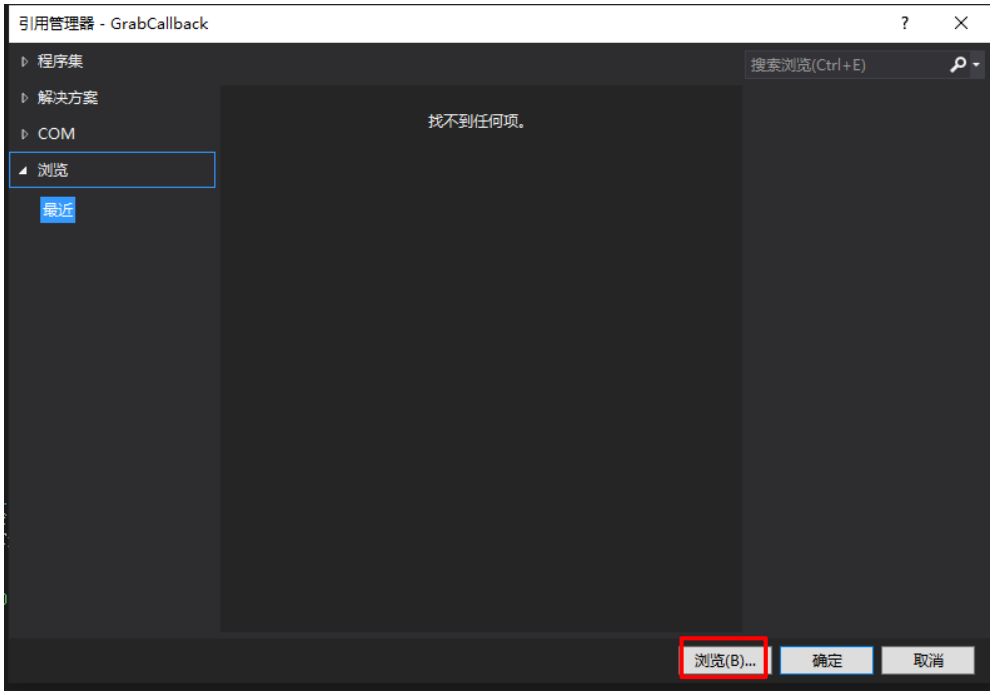
（1）拷贝 SDK 相关资源文件到工程目录中

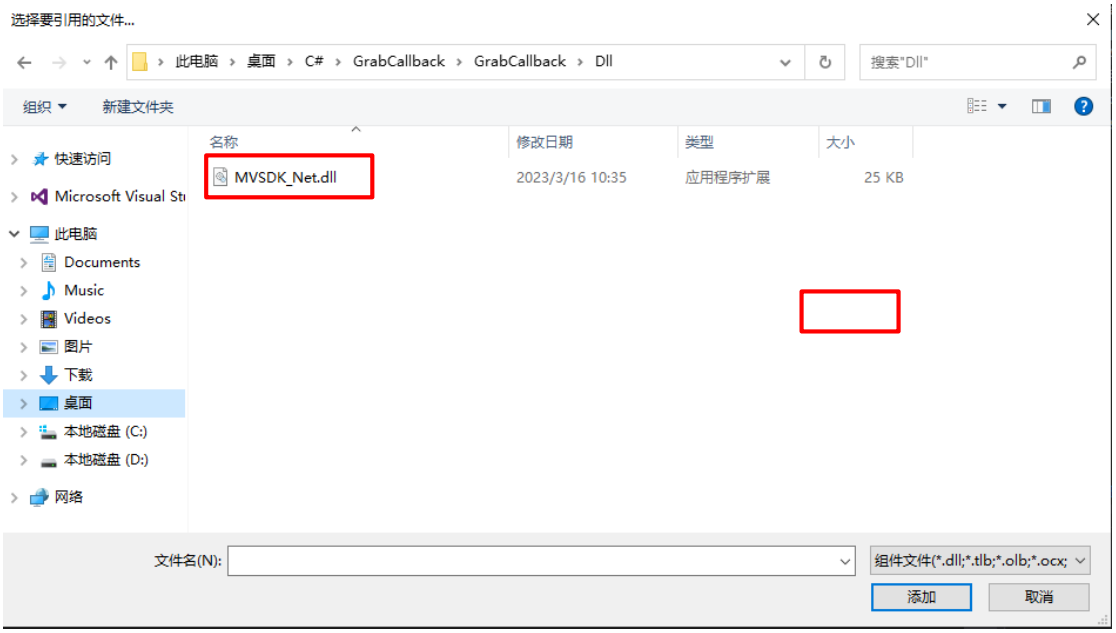


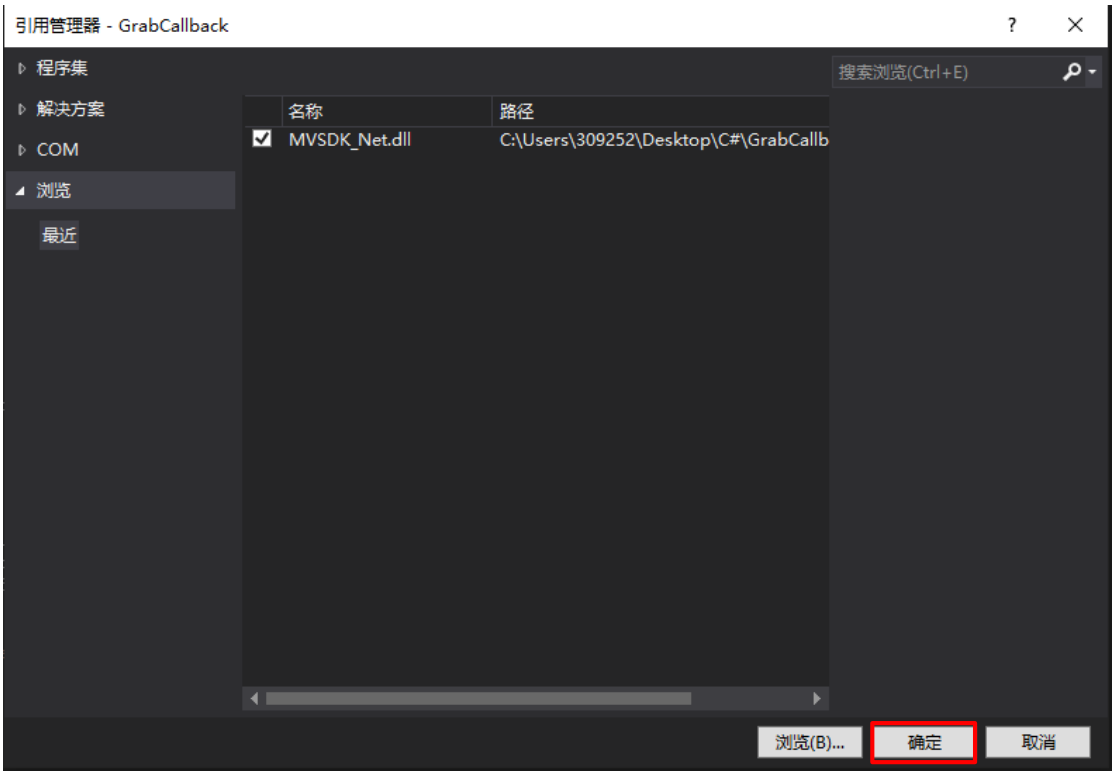
（2）配置工程

1、项目->添加引用，添加对 MVSDK_Net.dll 的引用；




引用管理器 - GrabCallback
程序集
解决方案
COM
浏览
最近
搜索浏览(Ctrl+E)
找不到任何项。
浏览(B)...
确定
取消

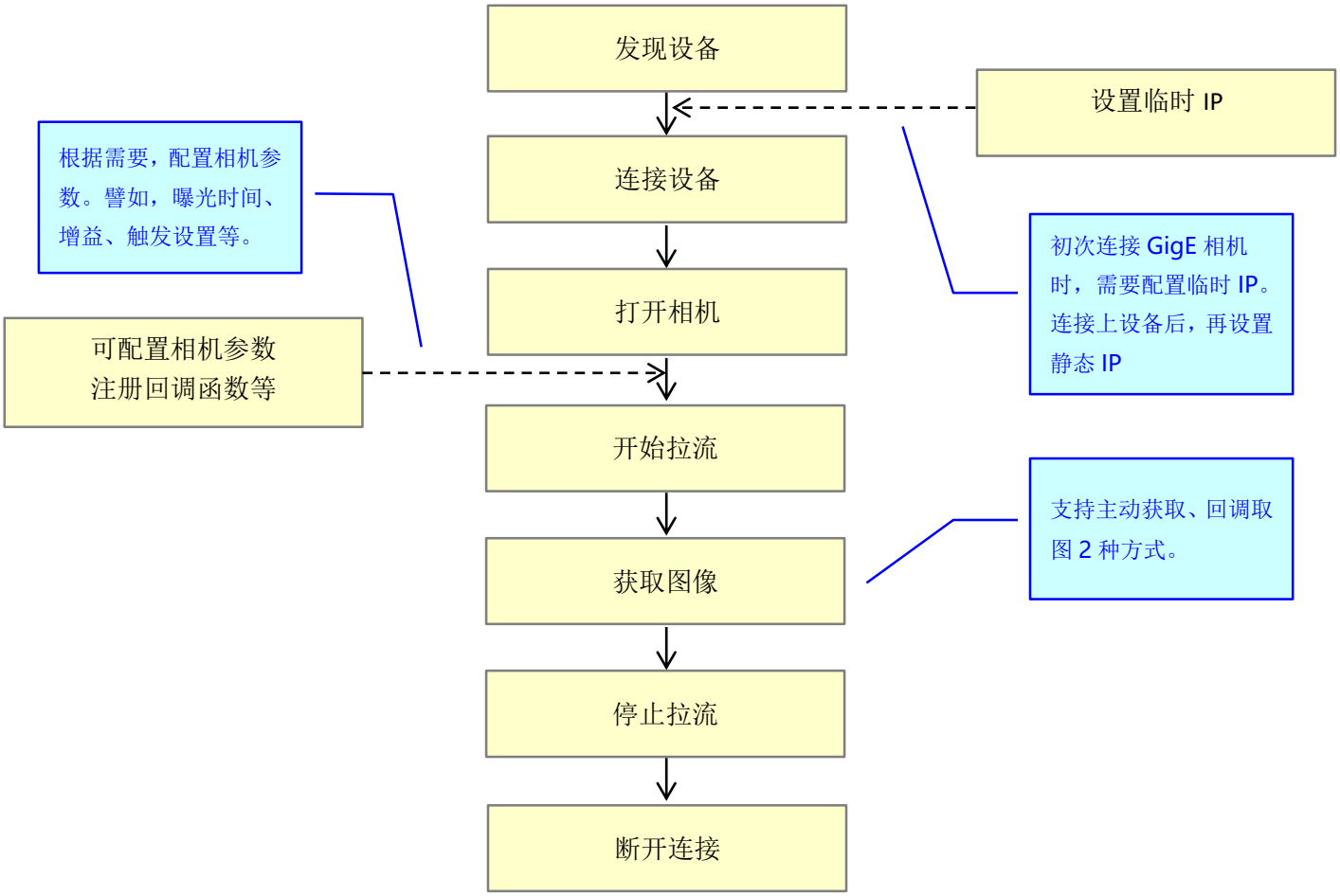

选择要引用的文件...
此电脑 › 桌面 › C# › GrabCallback › GrabCallback › Dll
搜索"Dll"
组织
新建文件夹
名称
修改日期
类型
大小
MVSDK_Net.dll
2023/3/16 10:35
应用程序扩展
25 KB
快速访问
Microsoft Visual St
此电脑
Documents
Music
Videos
图片
下载
桌面
本地磁盘 (C:)
本地磁盘 (D:)
网络
文件名(N):
组件文件(*.dll;*.tlb;*.olb;*.ocx;
添加
取消



2、在调用代码中添加：using MVSDK_Net;

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Runtime.InteropServices;
5  using System.Text;
6  using System.Threading;
7  using System.Threading.Tasks;
8  using MVSDK_Net;
```

2. SDK 的整体调用流程



3. SDK 的各功能接口调用说明

3.1.系统操作

3.1.1. 获取版本信息

接口: public static string IMV_GetVersion()
功能说明: 获取版本信息
成功时返回版本信息，失败时返回NULL
参考例程: 当前所有例程均有涉及

3.1.2. 枚举设备（发现设备）

接口: public static int IMV_EnumDevices(ref IMVDefine.IMV_DeviceList pDeviceList, UInt32 interfaceType);
功能说明: 枚举设备。
pDeviceList [OUT] 设备列表
interfaceType [IN] 待枚举的接口类型, 类型可任意组合,如 interfaceTypeGige | interfaceTypeUsb3
成功，返回IMV_OK，错误，返回错误码。
1、当interfaceType = interfaceTypeAll 时，枚举所有接口下的在线设备
2、当interfaceType = interfaceTypeGige 时，枚举所有GigE网口下的在线设备
3、当interfaceType = interfaceTypeUsb3 时，枚举所有USB接口下的在线设备
4、当interfaceType = interfaceTypeCL 时，枚举所有CameraLink接口下的在线设备
5、该接口下的interfaceType支持任意接口类型的组合,如，若枚举所有GigE网口和USB3接口下的在线设备时，可将interfaceType设置为 interfaceType = interfaceTypeGige | interfaceTypeUsb3,其它接口类型组合以此类推
参考例程: 当前所有例程均有涉及

接口: public static int IMV_EnumDevicesByUnicast(ref IMVDefine.IMV_DeviceList pDeviceList, string pIpAddress);
功能说明: 以单播形式枚举设备，仅限 Gige 设备使用。
pDeviceList [OUT] 设备列表
pIpAddress [IN] 设备的IP地址
成功，返回IMV_OK；错误，返回错误码。
参考例程: UnicastMode

3.1.3. 设备句柄的创建与销毁

接口: public int IMV_CreateHandle(IMVDefine.IMV_ECreateHandleMode mode,

```
int cameraIndex = 0, string cameraStr = "");
```

功能说明：通过指定标示符创建设备句柄，如指定索引、设备键、设备自定义名、IP地址。

mode [IN] 创建设备方式

cameraIndex [IN] 指定索引

cameraStr[IN]设备键、设备自定义名、IP地址

成功，返回IMV_OK，错误，返回错误码。

参考例程：当前所有例程均有涉及

接口：`public int IMV_DestroyHandle();`

功能说明：销毁设备句柄

成功，返回IMV_OK；错误，返回错误码。

参考例程：当前所有例程均有涉及

3.2.相机操作

接口：`public int IMV_GetDeviceInfo(ref IMVDefine.IMV_DeviceInfo pDevInfo);`

功能说明：获取设备信息

pDevInfo [OUT] 设备信息

成功，返回IMV_OK；错误，返回错误码

参考例程：当前所有例程均有涉及

3.2.1 相机的连接与关闭

接口：`public int IMV_Open();`

功能说明：打开设备

成功，返回IMV_OK；错误，返回错误码

参考例程：当前所有例程均有涉及

接口：`public int IMV_OpenEx(IMVDefine.IMV_ECameraAccessPermission accessPermission);`

功能说明：打开设备

accessPermission [IN] 控制通道权限(IMV_Open默认使用accessPermissionControl权限)

成功，返回IMV_OK；错误，返回错误码

与IMV_Open接口的区别在于连接操作执行时需判断访问权限

参考例程：暂无

接口：`public bool IMV_IsOpen();`

功能说明：判断设备是否已打开

打开状态，返回true；关闭状态或者掉线状态，返回false

参考例程：ForceIp

接口： public int IMV_Close();
功能说明：关闭设备
成功，返回IMV_OK；错误，返回错误码
参考例程：当前所有例程均有涉及

3.2.2 相机配置的下载、保存及加载配置

接口： public int IMV_DownLoadGenICamXML(string pFullFileName);
功能说明：下载设备描述XML文件，并保存到指定路径，如：D:\\xml.zip
pFullFileName [IN] 文件要保存的路径
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口： public int IMV_SaveDeviceCfg(string pFullFileName);
功能说明：保存设备配置到指定的位置。同名文件已存在时，覆盖。
pFullFileName [IN] 导出的设备配置文件全名(含路径)，如：D:\\config.xml 或 D:\\config.mvcfg
成功，返回IMV_OK；错误，返回错误码
参考例程：LoadAndSaveDeviceCfg

接口： public int IMV_LoadDeviceCfg(string pFullFileName,ref IMVDefine.IMV_ErrorList pErrorList);
功能说明：从文件加载设备xml配置
pFullFileName [IN] 设备配置(xml)文件全名(含路径)，如：D:\\config.xml 或 D:\\config.mvcfg
pErrorList [OUT] 加载失败的属性名列表。存放加载失败的属性上限为 IMV_MAX_ERROR_LIST_NUM。成功，返回IMV_OK；错误，返回错误码
参考例程：LoadAndSaveDeviceCfg

3.2.3 相机配置设置（GigE 相机特有）

接口： public int IMV_GIGE_ForceIpAddress(string pIpAddress,string pSubnetMask,string pGateway);
功能说明：修改设备IP，仅限Gige设备使用
pIpAddress [IN] IP地址
pSubnetMask [IN] 子网掩码
pGateway [IN] 默认网关
成功，返回IMV_OK；错误，返回错误码
1、调用该函数时如果pSubnetMask和pGateway都设置了有效值，则以此有效值为准；
2、调用该函数时如果pSubnetMask和pGateway都设置了NULL，则内部实现时用它所连接网卡的子网掩码和网关代替
3、调用该函数时如果pSubnetMask和pGateway两者中其中一个为NULL，另一个非NULL，则返回错误

参考例程：ForceIp

接口：　public in IMV_GIGE_GetAccessPermission(ref IMVDefine.IMV_ECameraAccessPermission pAccessPermission);
功能说明：获取设备的当前访问权限，仅限Gige设备使用
pAccessPermission [OUT] 设备的当前访问权限
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口：　public int IMV_GIGE_SetAnswerTimeout(UInt32 timeout);
功能说明：设置设备对sdk命令的响应超时时间,仅限Gige设备使用
timeout [IN] 超时时间，单位ms
成功，返回IMV_OK；错误，返回错误码
参考例程：GigECommunicationControl

3.2.4 本品牌相机特有功能

接口：　public int IMV_WriteUserPrivateData(IntPtr pBuffer,ref UInt32 pLength);
功能说明：写用户自定义数据。相机内部保留32768字节用于用户存储自定义数据(此功能针对本品牌相机，其它品牌相机无此功能)
pBuffer [IN] 数据缓冲的指针
pLength [IN] 期望写入的字节数 [OUT] 实际写入的字节数
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口：　public int IMV_ReadUserPrivateData(IntPtr pBuffer, ref UInt32 pLength);
功能说明：读用户自定义数据。相机内部保留32768字节用于用户存储自定义数据(此功能针对本品牌相机，其它品牌相机无此功能)
pBuffer [OUT] 数据缓冲的指针
pLength [IN] 期望读出的字节数 [OUT] 实际读出的字节数
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口：　public int IMV_WriteUARTData(IntPtr pBuffer, ref UInt32 pLength);
功能说明：往相机串口寄存器写数据，每次写会清除掉上次的数据(此功能只支持包含串口功能的本品牌相机)
pBuffer [IN] 数据缓冲的指针
pLength [IN] 期望写入的字节数 [OUT] 实际写入的字节数
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口：　public int IMV_ReadUARTData(IntPtr pBuffer, ref UInt32 pLength);
功能说明：从相机串口寄存器读取串口数据(此功能只支持包含串口功能的本品牌相机)
pBuffer [OUT] 数据缓冲的指针

pLength [IN] 期望读出的字节数 [OUT] 实际读出的字节数
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

3.3.注册事件回调

接口： public int IMV_SubscribeConnectArg(IMVDefine.IMV_ConnectCallBack proc, IntPtr pUser);
功能说明：设备连接状态事件回调注册
proc [IN] 设备连接状态事件回调函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：ResumeConnect

接口： public int IMV_SubscribeParamUpdateArg (IMVDefine.IMV_ParamUpdateCallBack proc, IntPtr pUser);
功能说明：参数更新事件回调注册
proc [IN] 参数更新注册的事件回调函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：Events

接口： public int IMV_SubscribeStreamArg(IMVDefine.IMV_StreamCallBack proc, IntPtr pUser);
功能说明：流通道事件回调注册
proc [IN] 流通道事件回调注册函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：Events

接口： public int IMV_SubscribeMsgChannelArg(IMVDefine.IMV_MsgChannelCallBack proc, IntPtr pUser);
功能说明：消息通道事件回调注册
proc [IN] 消息通道事件回调注册函数
pUser [IN] 用户自定义数据，可设为NULL
成功，返回IMV_OK；错误，返回错误码
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：Events

3.4.相机流资源操作

3.4.1. 流数据设置

接口： public int IMV_SetBufferCount(UInt32 nSize);
功能说明：设置帧数据缓存个数
nSize [IN] 缓存数量
成功，返回IMV_OK；错误，返回错误码
不能在拉流过程中设置
参考例程：暂无

接口： public int IMV_ClearFrameBuffer();
功能说明：清除帧数据缓存
成功，返回IMV_OK；错误，返回错误码
参考例程：ClearFrameBuffer

接口： public int IMV_GIGE_SetInterPacketTimeout(UInt32 nTimeout);
功能说明：设置驱动包间隔时间(MS),仅对Gige设备有效
nTimeout [IN] 包间隔时间，单位是毫秒
成功，返回IMV_OK；错误，返回错误码
触发模式尾包丢失重传机制
参考例程：GigECommunicationControl

接口： public int IMV_GIGE_SetSingleResendMaxPacketNum(UInt32 maxPacketNum);
功能说明：设置单次重传最大包个数，仅对GigE设备有效
maxPacketNum [IN] 单次重传最大包个数
成功，返回IMV_OK；错误，返回错误码
maxPacketNum为0时，该功能无效
参考例程：GigECommunicationControl

接口： public int IMV_GIGE_SetMaxLostPacketNum(UInt32 maxLostPacketNum);
功能说明：设置同一帧最大丢包的数量,仅对GigE设备有效
maxLostPacketNum [IN] 最大丢包的数量
成功，返回IMV_OK；错误，返回错误码
maxLostPacketNum为0时，该功能无效
只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册
参考例程：GigECommunicationControl

接口： public int IMV_USB_SetUrbTransfer(UInt32 nNum, UInt32 nSize);
功能说明：设置U3V设备的传输数据块的数量和大小,仅对USB设备有效
nNum [IN] 传输数据块的数量(范围:5-256)
nSize [IN] 传输数据块的大小(范围:8-512, 单位:KByte)
成功，返回IMV_OK；错误，返回错误码

1、该接口暂时只有在Linux平台且使用无驱的情况下设置才有效，其他情况下返回IMV_NOT_SUPPORT错误码

2、传输数据块数量，范围5 - 256，默认为64，该值设置过小会影响传输效率；多台相机共同使用时，可以适当减小该值

3、传输每个数据块大小，范围8 - 512，默认为64，单位是KByte

参考例程：暂无

3.4.2. 图像采集

接口：public int IMV_StartGrabbing();

功能说明：开始取流

成功，返回IMV_OK；错误，返回错误码

参考例程：Grab

接口：public int IMV_StartGrabbingEx(UInt64 maxImagesGrabbed, IMVDefine.IMV_EGrabStrategy strategy);

功能说明：开始取流

maxImagesGrabbed [IN] 允许最多的取帧数，达到指定取帧数后停止取流，如果为0，表示忽略此参数连续取流(IMV_StartGrabbing默认0)

strategy [IN] 取流策略,(IMV_StartGrabbing默认使用grabStrartegySequential策略取流)

成功，返回IMV_OK；错误，返回错误码

参考例程：GrabStrategy

接口：public bool IMV_IsGrabbing();

功能说明：判断设备是否正在取流

正在取流，返回true；不在取流，返回false

参考例程：MultipleCamera

接口：public int IMV_StopGrabbing();

功能说明：停止取流

成功，返回IMV_OK；错误，返回错误码

参考例程：Grab

3.4.3. 获取图像数据

接口：public int IMV_AttachGrabbing(IMVDefine.IMV_FrameCallBack proc, IntPtr pUser);

功能说明：注册帧数据回调函数(异步获取帧数据机制)

proc [IN] 帧数据信息回调函数，建议不要在该函数中处理耗时的操作，否则会阻塞后续帧数据的实时性

pUser [IN] 用户自定义数据，可设为NULL

成功，返回IMV_OK；错误，返回错误码

该异步获取帧数据机制和同步获取帧数据机制(IMV_GetFrame)互斥，对于同一设备，系统中两者只能选其一

只支持一个回调函数，且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：ChunkData、Events、FrameClone、FrameTriggerCount、GrabCallback

接口： public int IMV_GetFrame(ref IMVDefine.IMV_Frame pFrame, UInt32 timeoutMS);

功能说明：获取一帧图像(同步获取帧数据机制)

pFrame [OUT] 帧数据信息

timeoutMS [IN] 获取一帧图像的超时时间,INFINITE时表示无限等待,直到收到一帧数据或者停止取流。单位是毫秒

成功，返回IMV_OK；错误，返回错误码

该接口不支持多线程调用。

该同步获取帧机制和异步获取帧机制(IMV_AttachGrabbing)互斥,对于同一设备，系统中两者只能选其一。

使用内部缓存获取图像，需要IMV_ReleaseFrame进行释放图像缓存。

参考例程：ClearFrameBuffer、Grab

接口： public int IMV_ReleaseFrame(ref IMVDefine.IMV_Frame pFrame);

功能说明：释放图像缓存

pFrame [IN] 帧数据信息

成功，返回IMV_OK；错误，返回错误码

参考例程：ClearFrameBuffer、Grab

3.4.4. 流与图像数据的其他操作

接口： public int IMV_CloneFrame(ref IMVDefine.IMV_Frame pFrame,
ref IMVDefine.IMV_Frame pCloneFrame);

功能说明：帧数据深拷贝克隆

pFrame [IN] 克隆源帧数据信息

pCloneFrame [OUT] 新的帧数据信息

成功，返回IMV_OK；错误，返回错误码

使用IMV_ReleaseFrame进行释放图像缓存。

参考例程：FrameClone

接口： public int IMV_GetChunkDataByIndex(ref IMVDefine.IMV_Frame pFrame, UInt32 index,
ref IMVDefine.IMV_ChunkDataInfo pChunkDataInfo);

功能说明：获取Chunk数据(仅对GigE/Usb相机有效)

pFrame [IN] 帧数据信息

index [IN] 索引ID

pChunkDataInfo [OUT] Chunk数据信息

成功，返回IMV_OK；错误，返回错误码

参考例程：ChunkData

接口: public int IMV_GetStatisticsInfo(ref IMVDefine.IMV_StreamStatisticsInfo pStreamStatsInfo);
功能说明: 获取流统计信息(IMV_StartGrabbing / IMV_StartGrabbing执行后调用)
pStreamStatsInfo [OUT] 流统计信息数据
成功，返回IMV_OK；错误，返回错误码
参考例程: StreamStatistics

接口: public int IMV_ResetStatisticsInfo();
功能说明: 重置流统计信息(IMV_StartGrabbing / IMV_StartGrabbing执行后调用)
成功，返回IMV_OK；错误，返回错误码
参考例程: StreamStatistics

3.5.属性操作

3.5.1. 属性的判断、获取、设置

① 属性的判断
接口: public bool IMV_FeatureIsAvailable(string pFeatureName);
功能说明: 判断属性是否可用
pFeatureName [IN] 属性名
可，返回true；不可，返回false
参考例程: 暂无

接口: public bool IMV_FeatureIsReadable (string pFeatureName);
功能说明: 判断属性是否可读
pFeatureName [IN] 属性名
可，返回true；不可，返回false
参考例程: 暂无

接口: public bool IMV_FeatureIsWriteable (string pFeatureName);
功能说明: 判断属性是否可写
pFeatureName [IN] 属性名
可，返回true；不可，返回false
参考例程: 暂无

接口: public bool IMV_FeatureIsStreamable (string pFeatureName);
功能说明: 判断属性是否可流
pFeatureName [IN] 属性名
可，返回true；不可，返回false
参考例程: 暂无

接口: public bool IMV_FeatureIsValid (string pFeatureName);
功能说明: 判断属性是否有效

pFeatureName [IN] 属性名
可，返回true；不可，返回false
参考例程：Record

② 属性的获取
接口： public int IMV_GetIntFeatureValue(string pFeatureName, ref Int64 pIntValue);
功能说明：获取整型属性值
pFeatureName [IN] 属性名
pIntValue [OUT] 整型属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetIntFeatureMin (string pFeatureName, ref Int64 pIntValue);
功能说明：获取整型属性可设的最小值
pFeatureName [IN] 属性名
pIntValue [OUT] 整型属性可设的最小值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetIntFeatureMax (string pFeatureName, ref Int64 pIntValue);
功能说明：获取整型属性可设的最大值
pFeatureName [IN] 属性名
pIntValue [OUT] 整型属性可设的最大值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetIntFeatureInc (string pFeatureName, ref Int64 pIntValue);
功能说明：获取整型属性步长
pFeatureName [IN] 属性名
pIntValue [OUT] 整型属性步长
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetDoubleFeatureValue (string pFeatureName, ref double pDoubleValue);
功能说明：获取浮点属性值
pFeatureName [IN] 属性名
pIntValue [OUT] 浮点属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetDoubleFeatureMin (string pFeatureName, ref double pDoubleValue);
功能说明：获取浮点属性可设的最小值

pFeatureName [IN] 属性名
pIntValue [OUT] 浮点属性可设的最小值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetDoubleFeatureMax (string pFeatureName, ref double pDoubleValue);
功能说明：获取浮点属性可设的最大值
pFeatureName [IN] 属性名
pIntValue [OUT] 浮点属性可设的最大值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetBoolFeatureValue (string pFeatureName, ref bool pBoolValue);
功能说明：获取布尔属性值
pFeatureName [IN] 属性名
pIntValue [OUT] 布尔属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetStringFeatureValue (string pFeatureName, ref IMVDefine.IMV_String pStringValue);
功能说明：获取字符串属性值
pFeatureName [IN] 属性名
pIntValue [OUT] 字符串属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetEnumFeatureValue (string pFeatureName, ref UInt64 pEnumValue);
功能说明：获取枚举属性值
pFeatureName [IN] 属性名
pEnumValue [OUT] 枚举属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetEnumFeatureSymbol (string pFeatureName, ref IMVDefine.IMV_String pEnumValueSymbol);
功能说明：获取枚举属性symbol值
pFeatureName [IN] 属性名
pEnumValue [OUT] 枚举属性symbol值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetEnumFeatureEntryNum (string pFeatureName, ref UInt32 pEntryNum);
功能说明：获取枚举属性值可设置个数
pFeatureName [IN] 属性名
pEntryNum [OUT] 枚举属性值可设置个数
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_GetEnumFeatureEntrys (string pFeatureName, ref IMVDefine.IMV_EnumEntryList pEnumEntryList);
功能说明：获取枚举属性值可设置枚举值列表
pFeatureName [IN] 属性名
pEnumEntryList [OUT] 枚举属性值可设置枚举值列表
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

③ 属性的设置
接口： public int IMV_SetIntFeatureValue(string pFeatureName, Int64 intValue);
功能说明：设置整型属性值
pFeatureName [IN] 属性名
intValue [IN] 待设置的整型属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_SetDoubleFeatureValue (string pFeatureName, double doubleValue);
功能说明：设置浮点属性值
pFeatureName [IN] 属性名
doubleValue [IN] 待设置的浮点属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口： public int IMV_SetBoolFeatureValue (string pFeatureName, bool boolValue);
功能说明：设置布尔属性值
pFeatureName [IN] 属性名
boolValue [IN] 待设置的布尔属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：ChunkData

接口： public int IMV_SetEnumFeatureValue (string pFeatureName, UInt64 enumValue);
功能说明：设置枚举属性值
pFeatureName [IN] 属性名
enumValue [IN] 待设置的枚举属性值
成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

接口：　public int IMV_SetEnumFeatureSymbol (string pFeatureName, string pEnumSymbol);
功能说明：设置枚举属性symbol值
　　pFeatureName［IN］属性名
　　boolValue［IN］待设置的枚举属性symbol值
　　成功，返回IMV_OK；错误，返回错误码
参考例程：ChunkData、CommPropAccess、SoftTrigger、UserSetControl

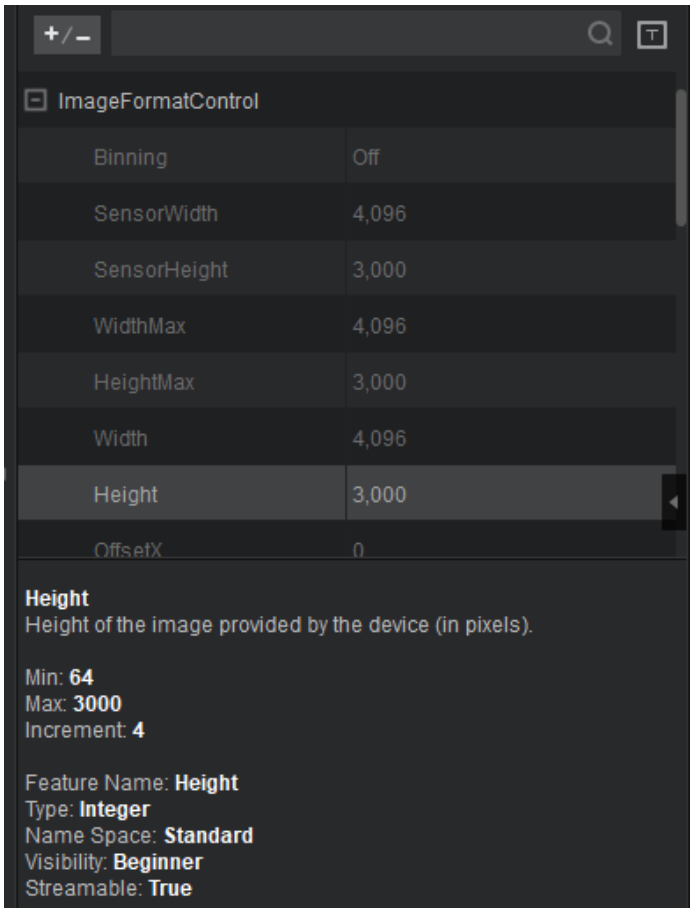
接口：　public int IMV_SetStringFeatureValue (string pFeatureName, string pStringValue);
功能说明：设置字符串属性值
　　pFeatureName［IN］属性名
　　pStringValue［IN］待设置的字符串属性值
　　成功，返回IMV_OK；错误，返回错误码
参考例程：CommPropAccess

3.5.2. 命令属性的执行

接口：　public int IMV_ExecuteCommandFeature(string pFeatureName);
功能说明：　执行命令属性
　　pFeatureName［IN］属性名
　　成功，返回IMV_OK；错误，返回错误码
参考例程：ChunkData、ClearFrameBuffer、 SoftTrigger、UserSetControl

3.5.3. 属性的查询与使用

打开相机客户端可查询对应的属性，并且可以了解属性的详细信息，如：数据类型、范围等。
1)　整型属性
例：若需获取相机的像素高度信息，找到 Height 属性并选中，下方会显示该属性的数据类型，根据 Type 的显示而定，下图 Height 属性的数据类型为 int 整型。



随后对所找到属性进行获取及修改。

```
int ret = IMVDefine.IMV_OK;
Int64 heightValue = 0;
Int64 heightMinValue = 0;
Int64 heightMaxValue = 0;
Int64 heightMaxValue = 0;
Mycamera cam=new Mycamera();

// 获取属性值
ret = cam.IMV_GetIntFeatureValue("Height", ref heightValue);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 获取属性可设的最小值，若无最小值则此操作错误
ret = cam.IMV_GetIntFeatureMin("Height", ref heightMinValue);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
    return ret;
```

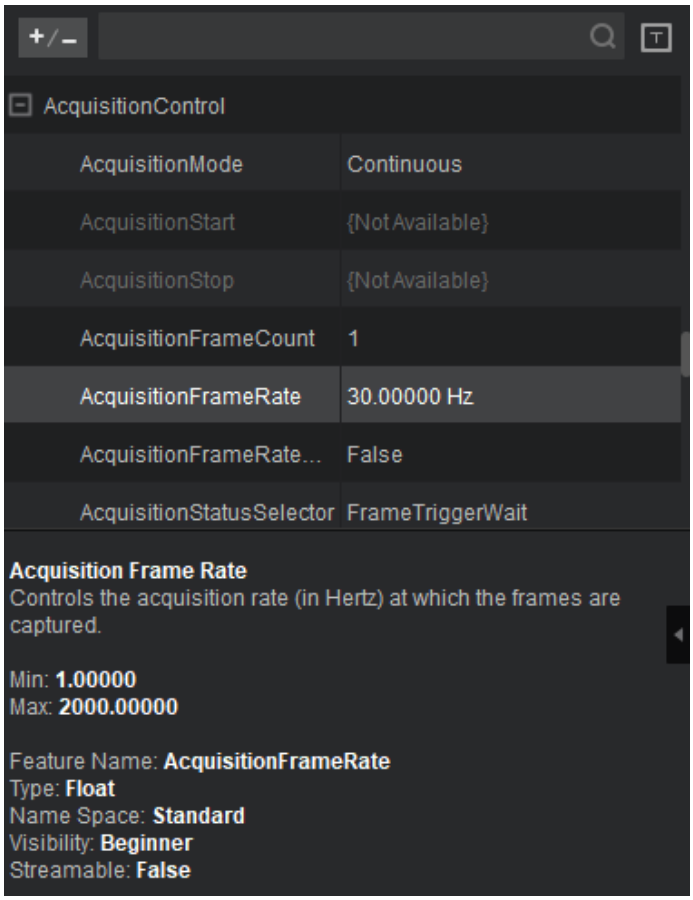
```
    }

    // 获取属性可设的最大值，若无最大值则此操作错误
    ret = cam.IMV_GetIntFeatureMax("Height", ref heightMaxValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
        return ret;
    }

    // 获取属性步长，若无步长值则此操作错误
    ret = cam.IMV_GetIntFeatureInc("Height", ref incrementValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Get feature increment value failed! ErrorCode[{ret}]");
        return ret;
    }

    // 设置属性值
    ret = cam.IMV_SetIntFeatureValue("Height", heightValue);
    if (IMVDefine.IMV_OK != ret)
    {
        Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
        return ret;
    }
```

2) 浮点型属性




```
int ret = IMVDefine.IMV_OK;
double framerate = 0;
double frameratemin = 0;
double frameratemax = 0;
Mycamera cam=new Mycamera();

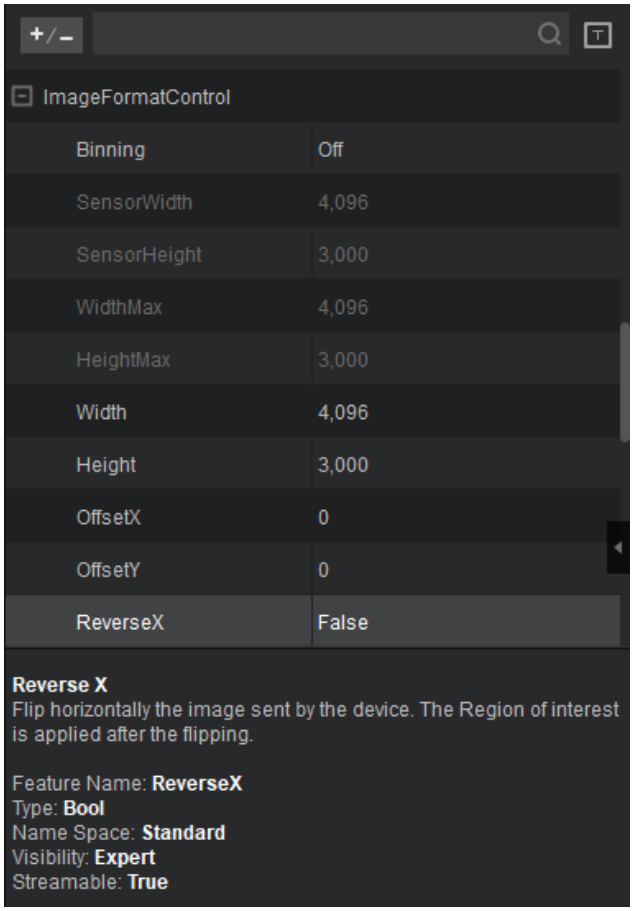
// 获取属性值
ret = cam.IMV_GetDoubleFeatureValue ("AcquisitionFrameRate", ref framerate);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}
// 获取属性可设的最小值，若无最小值则此操作错误
ret = cam.IMV_GetDoubleFeatureMin("AcquisitionFrameRate", ref frameratemin);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature minimum value failed! ErrorCode[{ret}]");
    return ret;
}

// 获取属性可设的最大值，若无最大值则此操作错误
```

```
ret = cam.IMV_GetDoubleFeatureMax("AcquisitionFrameRate", ref frameratemax);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature maximum value failed! ErrorCode[{ret}]");
    return ret;
}

// 设置属性值
ret = cam.IMV_SetDoubleFeatureValue("AcquisitionFrameRate", framerate);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

3) 布尔属性



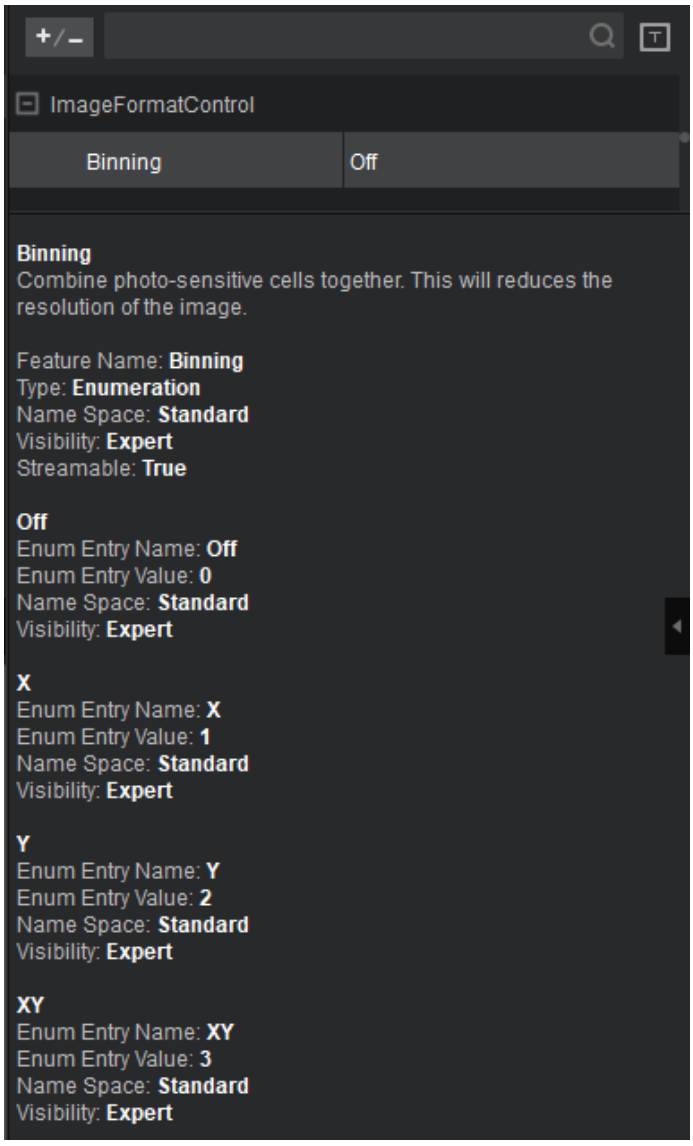

```
int ret = IMVDefine.IMV_OK;
bool reversevalue = 0;
Mycamera cam=new Mycamera();

// 获取属性值
ret = cam.IMV_GetBoolFeatureValue ("ReverseX", ref reversevalue);
if (IMVDefine.IMV_OK != ret)
```

```
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 设置属性值
ret = cam.IMV_SetBoolFeatureValue("ReverseX", reversevalue);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

4) 枚举属性




```
int ret = IMVDefine.IMV_OK;
UInt64 enumValue = 0;
UInt32 pEntryNum = 0;
```

```
IMVDefine.IMV_String pEnumSymbol=new IMVDefine.IMV_String();
IMVDefine.IMV_EnumEntryList pEnumEntryList=new IMVDefine.IMV_EnumEntryList();

// 获取属性值的可设枚举值列表
ret = cam.IMV_GetEnumFeatureValue("Binning", ref pEnumEntryList);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 获取属性可设枚举值的个数
ret = cam.IMV_GetEnumFeatureEntryNum("Binning", ref pEntryNum);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 获取属性symbol值
ret = cam.IMV_GetEnumFeatureSymbol("Binning", ref pEnumSymbol);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

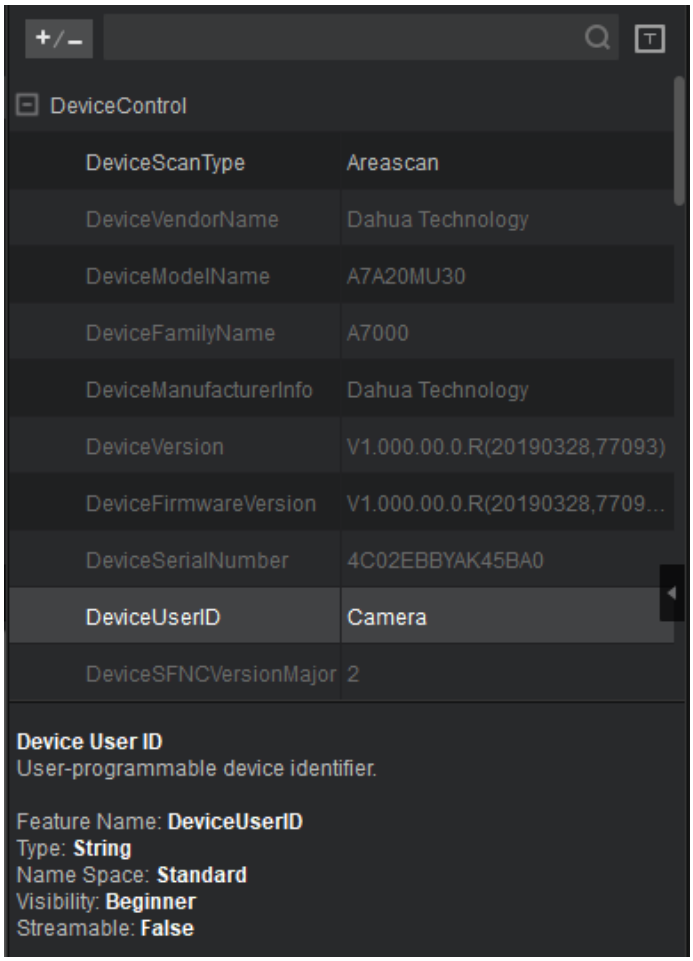
// 获取属性值
ret = cam.IMV_GetIntFeatureInc("Binning", ref enumValue);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 设置属性symbol值
ret = cam.IMV_SetEnumFeatureValue("Binning", pEnumSymbol);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 设置属性值
```

```
ret = cam.IMV_SetEnumFeatureValue("Binning", enumValue);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

5) 字符串属性



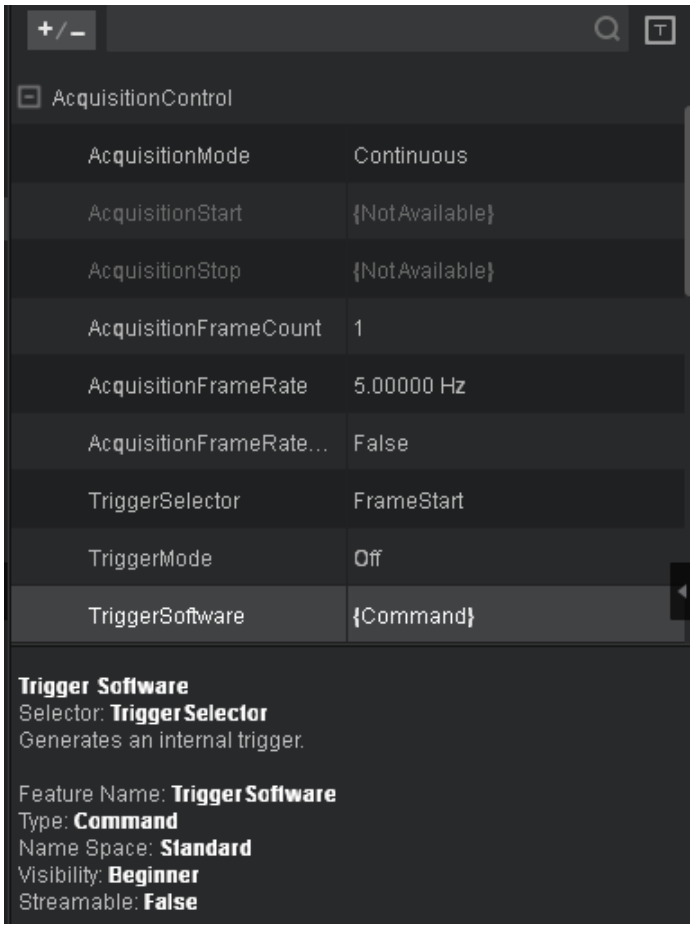

```
int ret = IMVDefine.IMV_OK;
string id = "";

// 获取属性值
ret = cam.IMV_GetStringFeatureValue ("DeviceUserID", ref id);
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Get feature value failed! ErrorCode[{ret}]");
    return ret;
}

// 设置属性值
ret = cam.IMV_SetStringFeatureValue("DeviceUserID", id);
```

```
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Set feature value failed! ErrorCode[{ret}]");
    return ret;
}
```

6) 命令属性




```
int ret = IMVDefine.IMV_OK;
// 执行命令属性值
ret = cam.IMV_ExecuteCommandFeature ("TriggerSoftware");
if (IMVDefine.IMV_OK != ret)
{
    Console.WriteLine($"Execute Command feature failed! ErrorCode[{ret}]");
    return ret;
}
```

3.6 图像操作

3.6.1 录像相关

接口： `public int IMV_OpenRecord(ref IMVDefine.IMV_RecordParam pstRecordParam);`

功能说明：打开录像
pstRecordParam [IN] 录像参数结构体
成功，返回IMV_OK；错误，返回错误码
参考例程：Record

接口：public int IMV_InputOneFrame(ref IMVDefine.IMV_RecordFrameInfoParam pstRecordFrameInfoParam);
功能说明：录制一帧图像
pstRecordFrameInfoParam [IN] 录像用帧信息结构体
成功，返回IMV_OK；错误，返回错误码
参考例程：暂无

接口：public int IMV_CloseRecord();
功能说明：关闭录像
成功，返回IMV_OK；错误，返回错误码
参考例程：Record

3.6.2 图像相关

接口：public int IMV_PixelConvert(ref IMVDefine.IMV_PixelConvertParam pstPixelConvertParam);
功能说明：像素格式转换
pstPixelConvertParam [IN][OUT] 像素格式转换参数结构体
成功，返回IMV_OK；错误，返回错误码
只支持转化成目标像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8 / gvspPixelBGRA8
通过该接口将原始图像数据转换成用户所需的像素格式并存放在调用者指定内存中。
像素格式为YUV411Packed的时，图像宽须能被4整除
像素格式为YUV422Packed的时，图像宽须能被2整除
像素格式为YUYVPacked的时，图像宽须能被2整除
转换后的图像:数据存储是从最上面第一行开始的，这个是相机数据的默认存储方向
参考例程：ImageConvert、SaveImageToBmp

接口：public int IMV_FlipImage(ref IMVDefine.IMV_FlipImageParam pstFlipImageParam);
功能说明：图像翻转
pstFlipImageParam [IN][OUT] 图像翻转参数结构体
成功，返回IMV_OK；错误，返回错误码
只支持像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8的图像的垂直和水平翻转。
通过该接口将原始图像数据翻转后并存放在调用者指定内存中。
参考例程：ImageFlip

接口：public int IMV_RotateImage(ref IMVDefine.IMV_RotateImageParam

pstRotateImageParam);

功能说明：图像顺时针旋转

pstRotateImageParam [IN][OUT] 图像旋转参数结构体

成功，返回IMV_OK；错误，返回错误码

只支持gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8格式数据的90/180/270度顺时针旋转。

通过该接口将原始图像数据旋转后并存放在调用者指定内存中。

参考例程：ImageRotate

接口： public int IMV_SaveImageToFile(ref IMVDefine.IMV_SaveImageToFileParam pstSaveImageToFileParam);

功能说明：图像保存到本地

pstSaveImageToFileParam [IN] 保存图片文件参数结构体

成功，返回IMV_OK；错误，返回错误码

该接口支持BMP/JPEG/PNG/TIFF

JPEG格式最大支持宽高为65500

参考例程：SaveImageToFile

4. 第三方平台的图像对象转换方法

4.1. Halcon 对象

4.1.1. 相机图像格式为 Mono8 时（主要是黑白格式）

Mono8 不需要转码，可以直接转换为 Halcon 图像对象。

这里假设 frame 是 SDK 取流得到的 IMV_Frame 数据帧结构体。

```
HObject Hobj; //Halcon 图像对象
HOperatorSet.GenImage1Extern(out Hobj,
                             "byte",
                             frame.frameInfo.width,
                             frame.frameInfo.height,
                             frame.pData,
                             IntPtr.Zero);
```

4.1.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）

非 Mono8 格式需要先转换为 BGR。

图像转码请参考 3.6.2 的图像格式转换。

这里假设 frame 是 SDK 取流得到的 IMV_Frame 数据帧结构体，pBGRbuffer 是转码之后的 BGR 数

据。

```
HObject Hobj; //Halcon 图像对象
HOperatorSet.GenImageInterleaved (out Hobj,
                        (HTuple) pBGRbuffer,
                        "bgr", frame.frameInfo.width,
                        frame.frameInfo.height, 0,
                        "byte", frame.frameInfo.width,
                        frame.frameInfo.height,0,0,8,0);
```

4.2. Mvp 图像对象（MvpImage）

4.2.1. 相机图像格式为 Mono8 时

Mono10、Mono12 等格式需参考 3.6.2 图像格式转换为 Mono8。相机图像格式为 Mono8 时，可直接用采集到的图像数据构建对象。

此处，假设 pFrame 为 SDK 取流得到的 IMV_Frame 数据帧结构体

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImage1(MvpImgDataType.MVP_IMGDTP_U8,
                    (int) pFrame.frameInfo.width,
                    (int)pFrame.frameInfo.height,
                    pData);
```

4.2.2. 相机图像格式为 Mono8 以外时（主要是彩色格式）

将采集到的相机图像数据，转换成 BGR24 格式(参考 Grab 例程 ConvertToBGR24),假设 pData 是转码之后的 BGR 数 据

使用转换后的 BGR24 格式图像数据，构建 Mvp 图像对象

```
MvpImage mvpimage = new MvpImage();
mvpimage.MvpGenImageInterleaved(pData,
                                EColorSpace.MVP_COLOR_SPACE_BGR,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                MvpImgDataType.MVP_IMGDTP_U8,
                                (int)pFrame.frameInfo.width,
                                (int)pFrame.frameInfo.height,
                                0,
                                0,
                                -1,
                                0);
```

5. 配套例程说明

C#例程路径：【SDK 安装目录】\Development\Samples\C#\IMV

例程名	例程说明
ChunkData	获取块数据
ClearFrameBuffer	清除图像缓存
CommPropAccess	通用属性使用例程
ConnectSpecCamera	连接指定的相机
Events	事件处理机制
ForceIp	修改相机 IP-仅限 GigE 相机需要
FrameClone	帧克隆
FrameTriggerCount	帧触发统计
GigECommunicationControl	GigE 通信控制-仅限 GigE 相机需要
Grab	自由模式采图（主动取流）
GrabCallback	拉流回调
GrabStrategy	拉流策略
ImageConvert	图像转化
ImageFlip	图像翻转
ImageRotate	图像顺时针旋转
LineTrigger	外触发方式采图（回调取流）
LoadAndSaveDeviceCfg	保存和加载相机属性配置文件
MultipleCamera	多相机拉流
ResumeConnect	断线重连
SaveImageToBmp	保存 BMP 图像
SaveImageToFile	保存图像到本地
SoftTrigger	软触发方式采图（回调取流）
StreamStatistics	流统计
UnicastMode	单播发现设备-仅限 GigE 设备
UserSetControl	保存和加载相机配置

6. 常见问题&注意点

6.1. 客户的程序使用 C#接口采图，只能取到 8 张图像（和 SDK 缓存个数一致）。

原因： C#接口取到图像对象后，没有释放，导致缓存被占满，无法再取到流。

解决方法：如下图所示，每取到帧 frame 都需要调用 IMV_ReleaseFrame 接口进行释放。

```
//取图
//get frame
IMVDefine.IMV_Frame frame = new IMVDefine.IMV_Frame();
res = cam.IMV_GetFrame(ref frame, 1000);
if (res != IMVDefine.IMV_OK)
{
    Console.WriteLine("Get frame failed! ErrorCode:[{0}]", res);
    break;
}

// 释放图像缓存
// Free image buffer
res = cam.IMV_ReleaseFrame(ref frame);
if (res != IMVDefine.IMV_OK)
{
    Console.WriteLine("Release frame failed! ErrorCode:[{0}]", res);
    break;
}
```

- END -